



Concurrent Programming

Week 7 (Lecture 1)

Stelios Tsampas

February 17, 2025

stelios@imada.sdu.dk

<https://www.steliotsampas.com>

stelios-tau/cp-2025



Stelios Tsampas
Responsible for DM584
stelios@imada.sdu.dk



Victor Gøth Ellefsen
Instructor
viell23@student.sdu.dk



Viktor Strate Kløvedal
Instructor
viktorstrate@imada.sdu.dk

Feel free to mail me, send me messages on Teams or drop by Ø14-602a-1.

Overview

Computers today have multi-core CPUs, and concurrency is essential for efficiency.

Many applications (e.g., web servers, databases, operating systems) rely on handling multiple tasks at once.

Understanding concurrency helps in writing efficient and bug-free programs.

Computers today have multi-core CPUs, and concurrency is essential for efficiency.

Many applications (e.g., web servers, databases, operating systems) rely on handling multiple tasks at once.

Understanding concurrency helps in writing efficient and bug-free programs.

Why do modern CPUs have more cores instead of just faster cores?

Why can a web server handle multiple users at once?

Concurrency

Concurrency is the ability of a system to handle **multiple tasks**, seemingly **simultaneously**. These tasks may **not** execute at the exact same time, but their execution is **interleaved**.

Concurrency

Concurrency is the ability of a system to handle **multiple tasks**, seemingly **simultaneously**. These tasks may **not** execute at the exact same time, but their execution is **interleaved**.

Non-deterministic interleaving is the key characteristic.

The programmer can make **no** assumptions as to the exact order of events.

May involve **shared resources**, requiring synchronization.

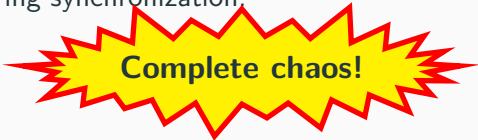
Concurrency

Concurrency is the ability of a system to handle **multiple tasks**, seemingly **simultaneously**. These tasks may **not** execute at the exact same time, but their execution is **interleaved**.

Non-deterministic interleaving is the key characteristic.

The programmer can make **no** assumptions as to the exact order of events.

May involve **shared resources**, requiring synchronization.



Complete chaos!

Programming without concurrency is challenging already.

Programming without concurrency is challenging already.

It gets even harder when you have to think about threads of execution being executed together, interacting with one another.

Programming without concurrency is challenging already.

It gets even harder when you have to think about threads of execution being executed together, interacting with one another.

We can tackle this new challenge by equipping ourselves with:

- Clean programming principles, and
- Tools for concurrent programming.

Techniques for concurrent software.

In Java!

Key concepts of concurrency.

Patterns and programming strategies for concurrency.

Lots of practice and hands-on coding, both by me and you.

Few of the lectures are taking place online through videos. Check itslearning to see which lectures are held in presence.

Links and videos will appear on itslearning.

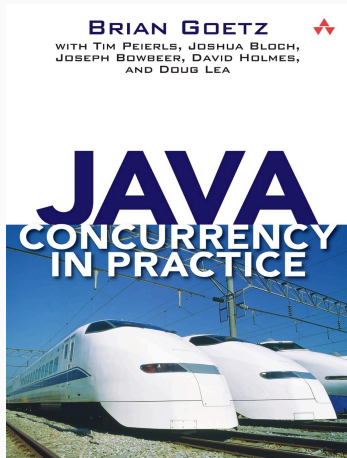
The code for the lectures and the exercises is on GitHub.

Book with theoretical aspects and code patterns.

Exercises every week.

Java Concurrency in Practice, *Brian Goetz et al.*, Addison-Wesley, 2006.

The book has a website,
<http://jcip.net/>.



General discussion conversation on itslearning.

I will start a Q&A thread in said conversation.

Would it make sense to make a discord server? The problem is not everybody has discord.

Software project (solo).

You will all solve the same problem.

Internal censor, 7-point scale.

The problem will be given during the course.

Follow the content posted on itslearning.

You can use whatever IDE/Code Editor that you like, both during the course and for developing the final software project, as long as you respect the final instructions for the exam when you hand in.

The course consists of two main parts:

Follow the content posted on itslearning.

You can use whatever IDE/Code Editor that you like, both during the course and for developing the final software project, as long as you respect the final instructions for the exam when you hand in.

The course consists of two main parts:

- Advanced Java, where we will cover some preliminaries about the Java language that are necessary for dealing well with concurrency.

- Concurrency, where we will deal with content specific to concurrency (principles, patterns, coding strategies).

Concurrency 101

Picture a chef (or more than one) cooking multiple dishes at the same time:

The chef prepares multiple meals at once by switching tasks.

Picture a chef (or more than one) cooking multiple dishes at the same time:

The chef prepares multiple meals at once by switching tasks.

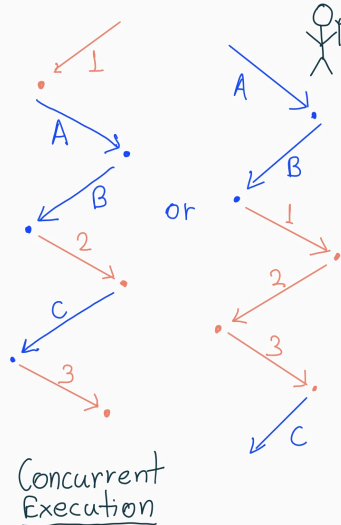
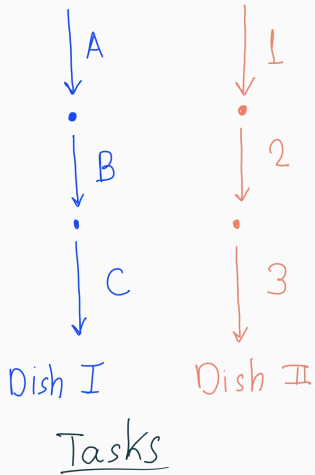
They might chop vegetables, then stir a sauce, then check the oven.

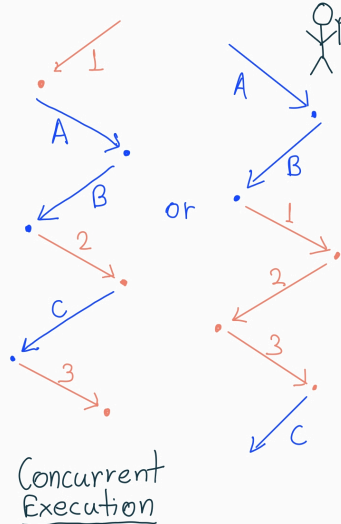
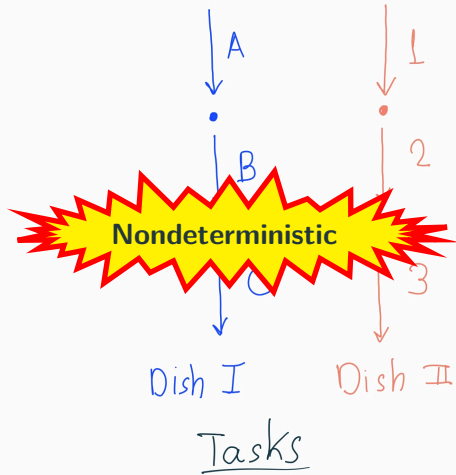
Picture a chef (or more than one) cooking multiple dishes at the same time:

- The chef prepares multiple meals at once by switching tasks.

- They might chop vegetables, then stir a sauce, then check the oven.

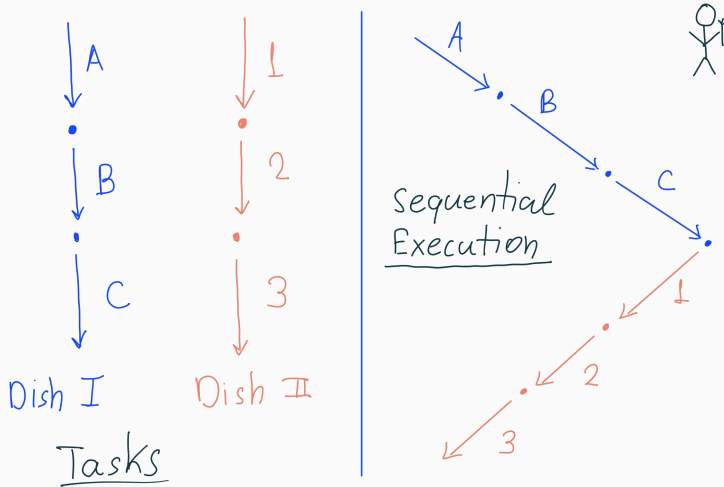
- The dishes are interleaved but not truly parallel.

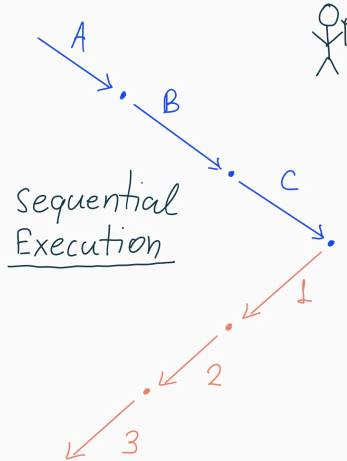
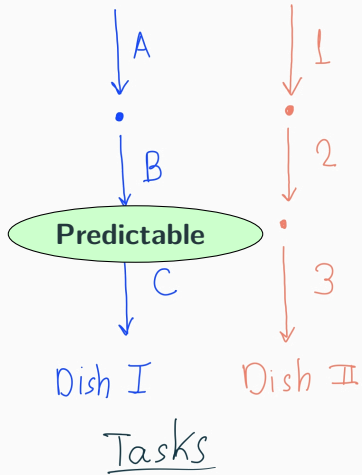




You don't need more than one chef/CPU cores/threads to have concurrent execution.

Task switching/interleaving are the hallmarks of concurrency.

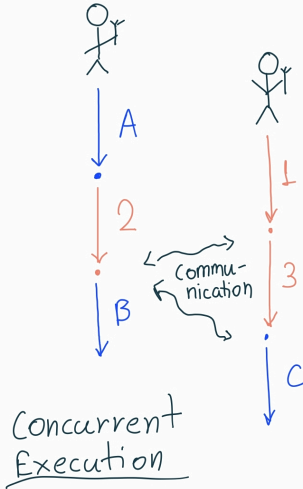
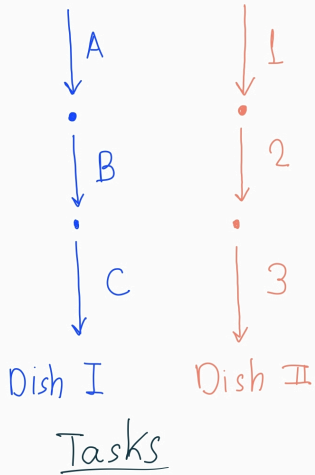


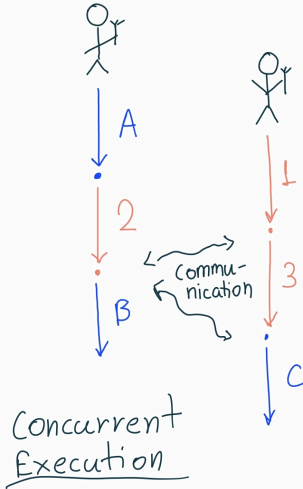
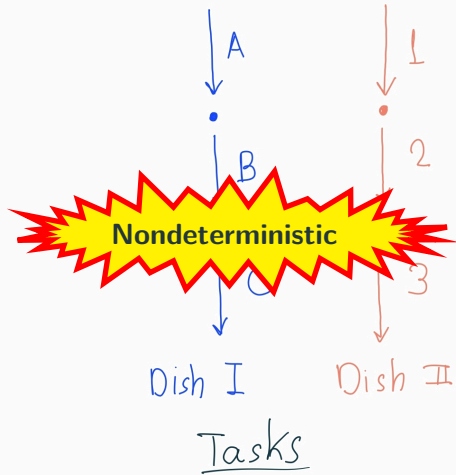


In sequential execution, tasks are *sequenced*, their execution will not be interrupted.

In sequential execution, tasks are *sequenced*, their execution will not be interrupted.

Simple and predictable, the kind of programming you have been doing so far.



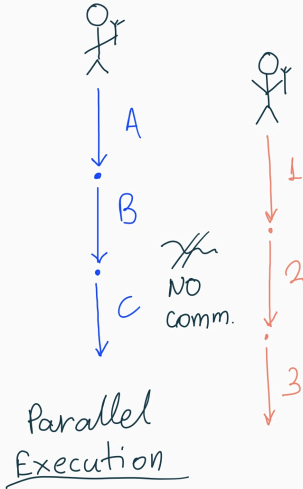
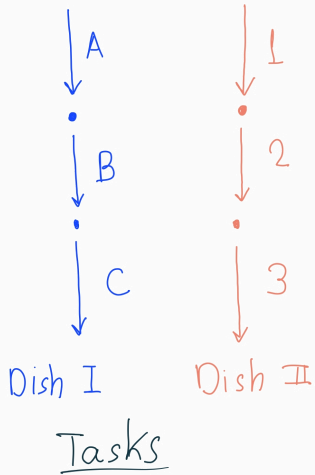


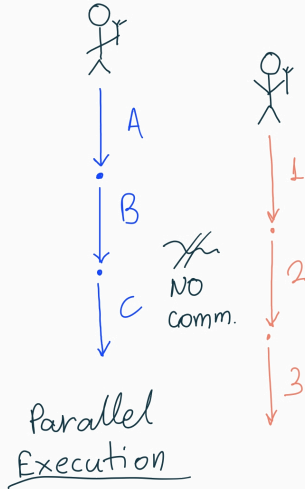
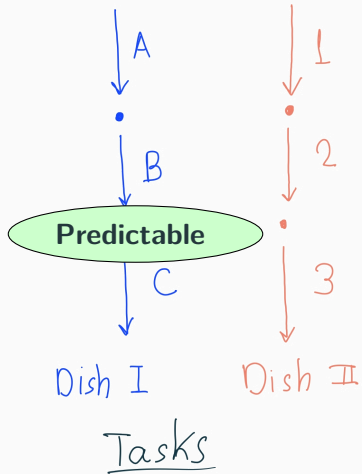
This time there are actually distinct chefs/**threads** of execution taking place.

This time there are actually distinct chefs/**threads** of execution taking place. They might be sharing resources (i.e. using the same equipment, food), so extra care needs to be taken, especially because the interleaving is **non-deterministic**.

This time there are actually distinct chefs/**threads** of execution taking place. They might be sharing resources (i.e. using the same equipment, food), so extra care needs to be taken, especially because the interleaving is **non-deterministic**.

Remember that these **threads** are abstractions, it is not necessary that, underneath the surface, there is a multi-core CPU executing each thread.





Again, there are two chefs/**threads** of execution taking place.

However, their tasks are distinct, and there is no communication between the two.

This is **parallel programming**, a different topic that is about the developing algorithms that make use of multiple CPU units.

Again, there are two chefs/**threads** of execution taking place.

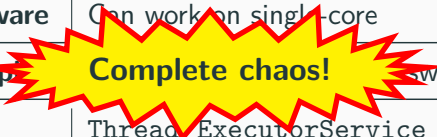
However, their tasks are distinct, and there is no communication between the two.

This is **parallel programming**, a different topic that is about the developing algorithms that make use of multiple CPU units.

Not quite the topic of this class, although the programming techniques involved may overlap.

Feature	Concurrency	Parallelism
Definition	Interleaved execution	True simultaneous execution
Execution	Task switching, may be single-core	Requires multiple cores
Use Case	Responsiveness (UI, servers)	Performance (big data, heavy comp.)
Hardware	Can work on single-core	Requires multi-core CPU
Example	Multi-threading with task switching	Multi-threading with multiple cores
Java	Thread, ExecutorService	ForkJoinPool, parallelStream()
Resources	Shared resources, synchronization	Little to no sharing and communication
Effect	Non-deterministic	(Largely) deterministic

Feature	Concurrency	Parallelism
Definition	Interleaved execution	True simultaneous execution
Execution	Task switching, may be single-core	Requires multiple cores
Use Case	Responsiveness (UI, servers)	Performance (big data, heavy comp.)
Hardware	Can work on single-core	Requires multi-core CPU
Example	Multi-threading with task switching	Multi-threading with multiple cores
Java	Thread, ExecutorService	ForkJoinPool, parallelStream()
Resources	Shared resources, synchronization	Little to no sharing and communication
Effect	Non-deterministic	(Largely) deterministic

Feature	Concurrency	Parallelism
Definition	Interleaved execution	True simultaneous execution
Execution	Task switching, may be single-core	Requires multiple cores
Use Case	Responsiveness (UI, servers)	Performance (big data, heavy comp.)
Hardware	Can work on single-core	Requires multi-core CPU
Example	Task switching  Complete chaos!	Multi-processor  Predictable
Java	ThreadPoolExecutor, ExecutorService	ForkJoinPool, parallelStream()
Resources	Shared resources, synchronization	Little to no sharing and communication
Effect	Non-deterministic	(Largely) deterministic

A large amount of this class is about managing the **sharing** of resources and generally the **communication** and **coordination** of these **threads** of execution.

A large amount of this class is about managing the **sharing** of resources and generally the **communication** and **coordination** of these **threads** of execution. Yes, the term *threads* is the technically correct term.

The main abstraction that is involved in concurrency.

A thread represents a lightweight, independent unit of execution.

Threads are present at multiple levels of the software stack:

- Operating system level, i.e. user and kernel threads.

- At the level of programming languages, Java threads, pthreads in C...

There can be many active threads active at the same time, i.e. *concurrently*.

The main abstraction that is involved in concurrency.

A thread represents a lightweight, independent unit of execution.

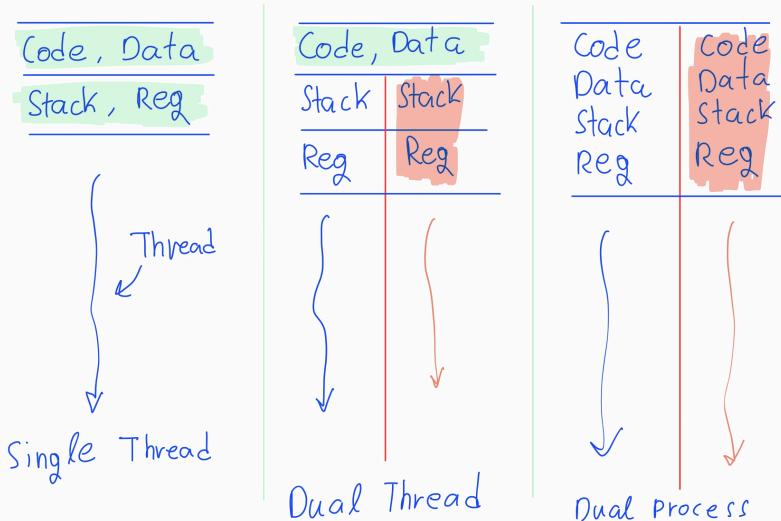
Threads are present at multiple levels of the software stack:

- Operating system level, i.e. user and kernel threads.

- At the level of programming languages, Java threads, pthreads in C...

There can be many active threads active at the same time, i.e. *concurrently*.

... but they do not need to run at the actual same moment.



Race conditions: Multiple threads accessing and modifying shared data inconsistently.

Deadlocks: Threads waiting indefinitely for each other.

Starvation: Some threads never get CPU time due to scheduling.

Visibility Issues: Changes in one thread not visible to others.

Race conditions: Multiple threads accessing and modifying shared data inconsistently.

Proper, careful synchronization.

Deadlocks: Threads waiting indefinitely for each other.

Proper, careful synchronization.

Starvation: Some threads never get CPU time due to scheduling.

Proper, careful synchronization.

Visibility Issues: Changes in one thread not visible to others.

Make no assumptions on data sharing between threads.

Thank you!